

Session 8 : Textures Avancées

💡 Objectifs de la session

Aller au-delà du chargement d'une image : générer des textures **dans le shader** (procédurales), rendre la scène **dans une texture** (render targets), simuler la profondeur avec des **parallax maps**, et se passer d'UVs avec le **triplanar mapping**. La session suit le même principe que la Session 7 : **créer dans Blender, intégrer dans Babylon.js / Node Material Editor**.

💡 Rappel Session 7

Une texture est une image sur le GPU, échantillonnée par le fragment shader via des UVs. On a vu le set PBR (albedo, normal, roughness, metallic, AO) et le filtrage (mipmaps, bilinear). Cette session explore ce qu'on peut faire **de plus** — quand une image fichier ne suffit pas, ou quand on n'a pas d'UVs.

Partie 1 : Textures Procédurales

Principe

Une texture procédurale est **calculée** dans le shader à partir des UVs (ou de la position) — pas lue dans un fichier image. Le shader devient la texture. Avantages :

- **Pas de mémoire** : zéro VRAM, quelle que soit la "résolution".
- **Résolution infinie** : pas de texel, pas de flou de près.
- **Paramétrable** : on change les paramètres du bruit, pas besoin de refaire une texture.

Inconvénient :

- **Coût shader** : chaque fragment exécute le calcul — un bruit complexe peut coûter plus qu'une lecture de texture.

1. Le bruit — la brique de base

💡 Pourquoi pas juste `random()` ?

Un générateur aléatoire pur (`rand()`) produit du **bruit blanc** : chaque pixel est indépendant de ses voisins. Visuellement, c'est de la neige TV — inutilisable pour une texture. Ce qu'on veut, c'est du bruit **cohérent** : des valeurs qui varient **progressivement** d'un pixel à l'autre, avec du détail à plusieurs échelles. C'est ce que font les fonctions de bruit procédural.

Le principe commun

Toutes les fonctions de bruit procédural partagent la même structure :

1. Une **grille** de points aléatoires (déterministes — même input → même output).
2. Une **interpolation** entre les points de la grille pour produire une valeur continue.

La différence entre les types de bruit est : **qu'est-ce qu'on interpole** et **comment**.

2. La fonction hash — le cœur du bruit

Hash déterministe

Un hash est une fonction qui prend un input (entier, vecteur) et retourne une valeur pseudo-aléatoire dans $[0, 1]$. **Déterministe** : le même input donne toujours le même output. C'est l'équivalent d'une table de nombres aléatoires pré-calculés, mais sans stockage.

```
// Hash 2D → 1 valeur [0,1]
float hash21(vec2 p) {
    vec3 p3 = fract(vec3(p.xy * 0.1031));
    p3 += dot(p3, p3.yzx + 33.33);
    return fract((p3.x + p3.y) * p3.z);
}

// Hash 2D → 2 valeurs [0,1] (pour les gradients)
vec2 hash22(vec2 p) {
    vec3 p3 = fract(vec3(p.xy * 0.1031, 0.1030, 0.0973));
    p3 += dot(p3, p3.yzx + 33.33);
    return fract((p3.xx + p3.yz) * p3.zy);
}
```

Les constantes magiques (0.1031, 33.33) sont empiriques — elles garantissent une bonne distribution pseudo-aléatoire. On n'invente pas les siennes ; on utilise des hash connus et testés.

🔗 Quel hash pour quel usage ?

- **Value noise** : n'importe quel hash (IQ, sin) suffit — on n'a besoin que d'une valeur déterministe.
- **Perlin / gradient noise** : le hash doit **décorréliser** les cellules voisines, sinon on voit la grille. Les hashes simples (IQ, sin) ne le font pas assez bien — on utilise une **table de permutation** (mod289 + permute), voir section 4.

3. Value Noise — le plus simple

Principe

On place une valeur aléatoire $[0, 1]$ à chaque point d'une grille entière. Pour un pixel entre les points, on interpole (bilinear + smoothstep) entre les 4 valeurs voisines.

$$\text{value}(p) = \text{interp}(\#(\text{floor}(p)), \#(\text{floor}(p) + (1, 0)), \#(\text{floor}(p) + (0, 1)), \#(\text{floor}(p) + (1, 1)))$$

Value noise en GLSL.

```
float valueNoise(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);
    // Smoothstep : adoucit les transitions (sinon bilinear = dur)
    f = f * f * (3.0 - 2.0 * f);

    float a = hash21(i);
    float b = hash21(i + vec2(1.0, 0.0));
    float c = hash21(i + vec2(0.0, 1.0));
    float d = hash21(i + vec2(1.0, 1.0));
```

```

    return mix(mix(a, b, f.x), mix(c, d, f.x), f.y);
}

```

⚠ Limites du value noise

Le value noise est **bloqueux** : on voit les cellules de la grille, surtout à basse fréquence. Les transitions sont en C^0 (continues mais pas dérivables) — la pente change brusquement aux bordures de cellules. Acceptable pour des effets simples, mais pas pour du réalisme.

4. Perlin Noise (gradient noise) — le standard

Principe

Inventé par **Ken Perlin** en 1985 (Oscar technique 1997). Au lieu d'interpoler des **valeurs**, on interpole des **gradients** — des vecteurs de direction aléatoires à chaque point de la grille. Pour un pixel, on calcule le produit scalaire entre le gradient et le vecteur du point de grille vers le pixel.

$$\text{perlin}(p) = \text{interp}(\text{dot}(g_A, p - A), \text{dot}(g_B, p - B), \text{dot}(g_C, p - C), \text{dot}(g_D, p - D))$$

où g_A, g_B, g_C, g_D sont les gradients aléatoires aux 4 coins de la cellule.

💡 Pourquoi Perlin > Value

Le Perlin noise est **C^1 -continu** : la dérivée est continue, donc pas de “cassure” visible. Les transitions sont douces et naturelles. C’est pourquoi c’est le bruit standard en infographie — tous les moteurs (Blender, Unity, Unreal, Substance) l’utilisent par défaut.

Perlin noise en GLSL (version pédagogique).

```

float perlinNoise(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);
    // Quintic smoothstep : C2-continu (encore plus doux que C1)
    f = f * f * f * (f * (f * 6.0 - 15.0) + 10.0);

    // Gradients aléatoires aux 4 coins – ATTENTION : il faut
    // (1) remapper [0,1] → [-1,1] (2) normaliser en vecteur unitaire
    // Sinon : tous les gradients pointent vers le même quadrant → artefacts.
    vec2 ga = normalize(hash22(i) * 2.0 - 1.0);
    vec2 gb = normalize(hash22(i + vec2(1.0, 0.0)) * 2.0 - 1.0);
    vec2 gc = normalize(hash22(i + vec2(0.0, 1.0)) * 2.0 - 1.0);
    vec2 gd = normalize(hash22(i + vec2(1.0, 1.0)) * 2.0 - 1.0);

    // Produits scalaires : gradient · vecteur vers le pixel
    float va = dot(ga, f - vec2(0.0, 0.0));
    float vb = dot(gb, f - vec2(1.0, 0.0));
    float vc = dot(gc, f - vec2(0.0, 1.0));
    float vd = dot(gd, f - vec2(1.0, 1.0));

    return mix(mix(va, vb, f.x), mix(vc, vd, f.x), f.y) * 0.5 + 0.5;
}

```

Le $* 0.5 + 0.5$ à la fin remappe le résultat de $[-1, 1]$ vers $[0, 1]$ (les produits scalaires peuvent être négatifs).

⚠ Cette version pédagogique a un défaut

Même avec des gradients normalisés, un hash simple (sin, IQ) produit des **lignes de grille** visibles à basse fréquence : les cellules voisines ont des gradients trop corrélés. C'est suffisant pour comprendre le principe, mais pas pour un rendu de production.

💡 La solution : table de permutation

Les implémentations réelles (Blender, Unity, la démo `session08_noise.html`) utilisent une **table de permutation** — un hash basé sur `mod289 + permute` qui décorrèle vraiment les cellules voisines. C'est l'implémentation de **Stefan Gustavson / Ashima Arts** (2005), devenue le standard GLSL. Le code est plus long mais le résultat est un vrai bruit de type nuage, sans artefacts. Voir `examples/session08_noise.js` pour le code complet.

5. Simplex Noise — l'évolution

Principe

Inventé par **Ken Perlin** en 2001 pour corriger les défauts du Perlin original. Au lieu d'une grille **carrée** (4 coins en 2D), on utilise une grille **triangulaire** (3 coins en 2D). Avantages :

- **Moins d'artefacts directionnels** : le Perlin a des "lignes" visibles le long des axes de la grille. Le Simplex n'en a pas.
- **Plus rapide en N dimensions** : le Perlin nécessite 2^N coins ; le Simplex n'en nécessite que $N + 1$. En 3D : 8 coins (Perlin) vs 4 (Simplex). En 4D : 16 vs 5.
- **Meilleure isotropie** : le bruit a la même apparence dans toutes les directions.

💡 Quand utiliser Simplex vs Perlin ?

- **Perlin** : plus simple à comprendre et implémenter. Suffisant pour la plupart des effets 2D.
- **Simplex** : préférable en 3D et 4D (volumetric, textures solides), ou quand on veut éviter les artefacts directionnels. L'implémentation est plus complexe (skewing de la grille triangulaire).

6. Worley Noise (cellular) — les cellules

Principe

Inventé par **Steven Worley** en 1996. Au lieu d'interpoler, on calcule la **distance** au point aléatoire le plus proche dans la grille. Résultat : un motif de **cellules** (type Voronoi) — chaque cellule contient un point, et la valeur est la distance au point de sa cellule.

$$\text{worley}(p) = \min_{\text{voisins}} \text{dist}(p, \text{point}_{\text{aléatoire}})$$

Worley noise en GLSL.

```
float worleyNoise(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);

    float minDist = 1.0;

    // Chercher dans les 9 cellules voisines (3x3)
```

```

    for (int y = -1; y <= 1; y++) {
        for (int x = -1; x <= 1; x++) {
            vec2 neighbor = vec2(float(x), float(y));
            vec2 point = hash22(i + neighbor);
            vec2 diff = neighbor + point - f;
            float dist = length(diff);
            minDist = min(minDist, dist);
        }
    }
    return minDist;
}

```

💡 Usage du Worley

Le Worley est idéal pour les motifs **cellulaires** : écailles de reptile, bulles, cellules organiques, pierre fracturée, écume. On peut aussi utiliser la distance au **deuxième** point le plus proche (F2-F1) pour produire des bordures nettes entre cellules.

7. FBM (Fractal Brownian Motion) — le détail à toutes les échelles

💬 Le problème du bruit simple

Un bruit seul (value, Perlin, simplex) a une **fréquence unique** — il est lisse ou détaillé, mais pas les deux. La nature a du détail à **toutes les échelles** : une montagne a des pics, des ravines, des rochers, des graviers, du sable. Une seule couche de bruit ne peut pas reproduire ça.

Principe du FBM

On superpose plusieurs **octaves** de bruit à des fréquences croissantes et amplitudes décroissantes :

$$\text{fbm}(p) = \sum_{i=0}^{N-1} a_i \cdot \text{noise}(f_i \cdot p)$$

avec :

- f_i = lacunaritéⁱ (fréquence — typiquement lacunarité = 2)
- a_i = persistanceⁱ (amplitude — typiquement persistance = 0.5)

Chaque octave ajoute du détail **plus fin** mais avec **moins d'amplitude**. Le résultat a du détail à toutes les échelles — comme la nature.

FBM en GLSL.

```

float fbm(vec2 p) {
    float value = 0.0;
    float amplitude = 0.5;
    float frequency = 1.0;

    for (int i = 0; i < 5; i++) {
        value += amplitude * perlinNoise(p * frequency);
        frequency *= 2.0; // lacunarité : double la fréquence
        amplitude *= 0.5; // persistance : divise l'amplitude par 2
    }
}

```

```

    return value;
}

```

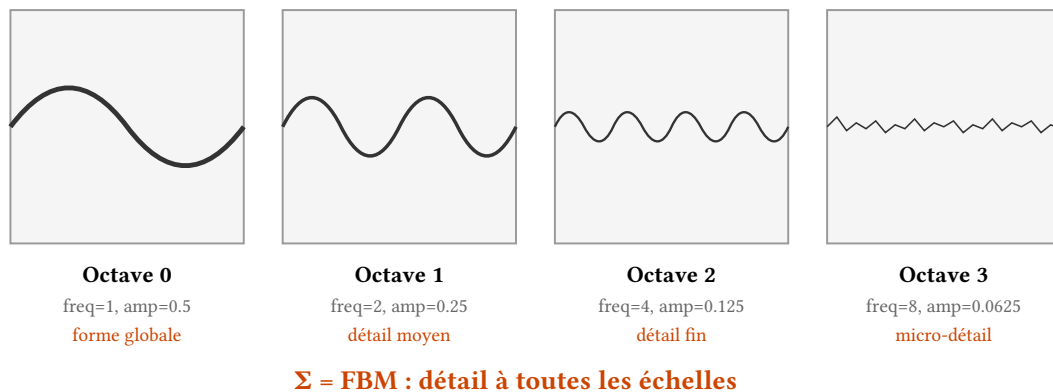


Figure 1: Les 4 octaves du FBM. Chaque octave double la fréquence (détail plus fin) et divise l’amplitude par 2 (contribution plus faible). La somme produit du détail à toutes les échelles.

Les deux paramètres clés

- **Lacunarité** (L) : facteur multiplicatif de la fréquence entre octaves. $L = 2$ (défaut) : chaque octave a le double de détail. $L > 2$: détail plus dense, plus “chaotique”. $L < 2$: détail plus espacé, plus “doux”.
- **Persistence** (P) : facteur multiplicatif de l’amplitude entre octaves. $P = 0.5$ (défaut) : chaque octave contribue moitié moins. $P > 0.5$: les hautes fréquences sont plus visibles (terrain rocheux). $P < 0.5$: les hautes fréquences sont atténuées (terrain doux).

8. Comparaison visuelle

Bruit	Continuité	Coût	Usage
Value	C^0 (bloqueux)	Faible	Prototypage, effets simples
Perlin	C^1 (doux)	Moyen	Standard : terrain, nuages, marbre
Simplex	C^1 (isotrope)	Moyen	3D/4D, pas d’artefacts directionnels
Worley	Discontinu	Moyen	Cellules : écailles, bulles, pierre
FBM	Dépend du bruit de base	×N octaves	Terrain réaliste, textures naturelles

Table 1: Les 5 types de bruit. Le FBM n’est pas un bruit indépendant — c’est une **combinaison** d’octaves d’un autre bruit (typiquement Perlin).

💡 Démo interactive

Ouvrir examples/session08_noise.html : une galerie interactive qui montre les 5 types de bruit sur un plan, avec une GUI pour :

- **Type de bruit** : Value, Perlin, Simplex, Worley, FBM.
- **Fréquence** : densité du bruit (zoom in/out).
- **Contraste / Luminosité** : post-traitement du résultat.
- **Animer** : déplace le bruit dans le temps.
- **Mode terrain** : applique le bruit comme height map avec coloration (eau → plaine → montagne → neige).

9. Applications procédurales

Le principe commun

Toutes ces textures combinent un bruit avec une **fonction de forme** (sinus, seuil, distance) pour produire un motif reconnaissable. Le bruit apporte le **chaos naturel**, la fonction de forme apporte la **structure**.

La démo exemples/session08_textures.html montre ces 5 textures en temps réel avec une GUI et un bouton d'export PNG.

Marbre

Principe

Des veines sinusoïdales perturbées par du FBM. Le FBM casse la régularité du sinus pour donner des veines naturelles et irrégulières.

Marbre en GLSL.

```
vec3 marble(vec2 uv) {
    float n = fbm(uv * 2.0);
    // Veines : sinus absolu pour des lignes nettes
    float vein = abs(sin((uv.x + n * 1.5) * 3.14159));
    // Couleur : blanc → gris-bleu foncé
    vec3 light = vec3(0.95, 0.95, 0.92);
    vec3 dark = vec3(0.30, 0.35, 0.45);
    return mix(dark, light, vein);
}
```

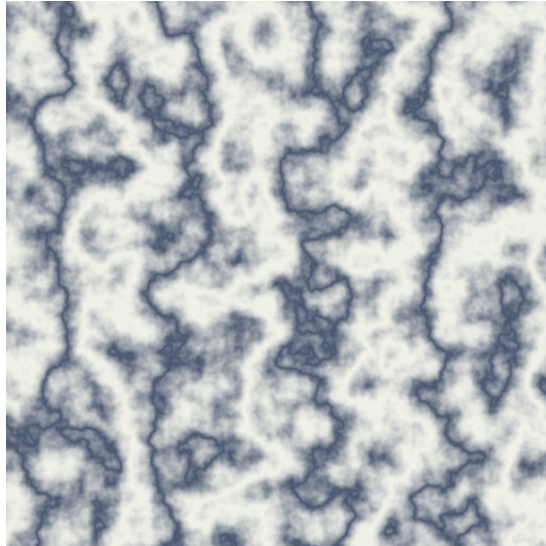


Figure 2: Texture de marbre procédurale — FBM + sinus absolu. Les veines sont irrégulières car le FBM perturbe la phase du sinus.

Bois

Principe

Des anneaux concentriques déformés par du bruit. Le `abs(sin(...))` crée des anneaux, le bruit les déforme pour qu'ils ne soient pas parfaits.

Bois en GLSL.

```
vec3 wood(vec2 uv) {  
    float n = perlinNoise(uv * 3.0);  
    // Anneaux : sinus absolu (effet de rings)  
    float rings = abs(sin((uv.x * 8.0 + n * 5.0) * 3.14159));  
    // Couleur : bois clair → bois foncé  
    vec3 light = vec3(0.80, 0.60, 0.35);  
    vec3 dark  = vec3(0.40, 0.25, 0.12);  
    return mix(dark, light, rings);  
}
```

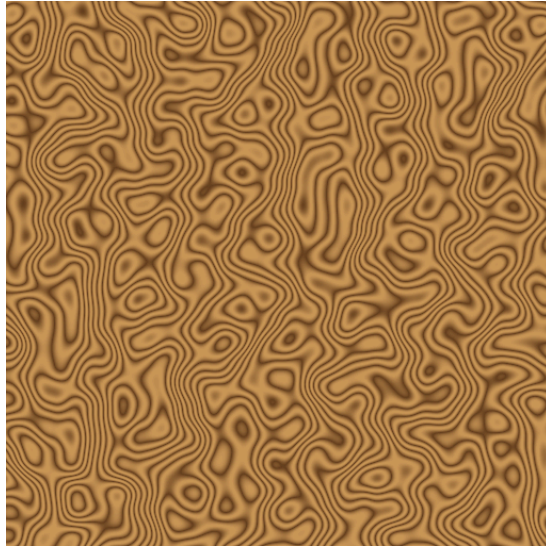


Figure 3: Texture de bois procédurale — Perlin + sinus absolu. Les anneaux sont déformés par le bruit, imitant les irrégularités du bois réel.

Dissolve

Principe

Un bruit seuillé — `step(threshold, noise(uv))` — pour faire fondre un objet. Animer le `threshold` de 0 à 1 fait disparaître l'objet progressivement. On ajoute souvent un **halo lumineux** au seuil pour un effet de "brûlure".

Dissolve en GLSL.

```
vec3 dissolve(vec2 uv) {  
    float n = fbm(uv * 3.0);  
    float mask = step(uThreshold, n);  
    // Bordure lumineuse au seuil (effet "burn")  
    float edge = smoothstep(uThreshold - 0.05, uThreshold, n)  
                - smoothstep(uThreshold, uThreshold + 0.05, n);  
    vec3 base = vec3(0.1, 0.1, 0.15);  
    vec3 solid = vec3(0.9, 0.9, 0.95);  
    vec3 color = mix(base, solid, mask);  
    color += edge * vec3(1.0, 0.5, 0.1); // halo orange  
    return color;  
}
```

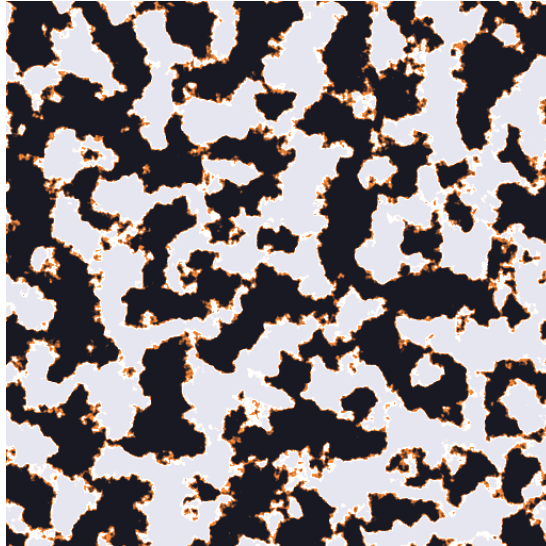


Figure 4: Effet dissolve — FBM seuillé avec halo orange au bord. Animer le seuil fait fondre l'objet progressivement, utile pour les transitions et la mort d'ennemis.

Écailles (Worley)

Principe

Le Worley noise avec un seuil crée des cellules séparées par des bordures — parfait pour les écailles de reptile, les cellules organiques, ou la pierre fracturée.

Écailles en GLSL.

```
vec3 scales(vec2 uv) {
    float w = worleyNoise(uv * 4.0);
    // Bordures : inverse du worley, seuillé
    float border = 1.0 - smoothstep(0.0, 0.15, w);
    // Couleur de base : dégradé selon la distance
    vec3 base = mix(vec3(0.2, 0.5, 0.6), vec3(0.4, 0.7, 0.8), w);
    // Bordures sombres
    return mix(base, vec3(0.05, 0.1, 0.15), border);
}
```

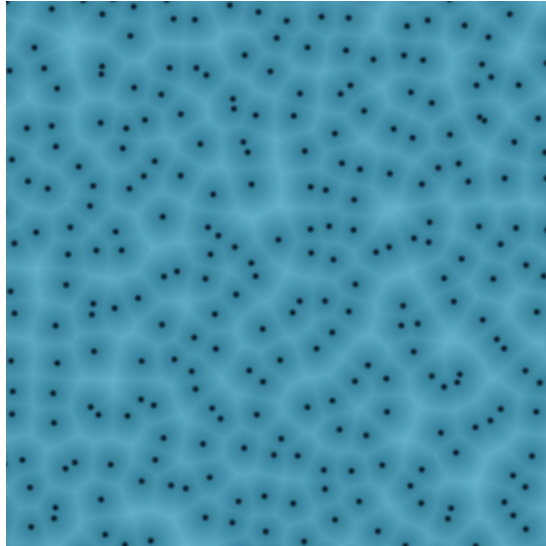


Figure 5: Texture d'écailles — Worley noise avec bordures sombres. Chaque cellule correspond à un point de la grille Voronoi. Idéal pour les reptiles, poissons, ou surfaces cellulaires.

Terrain

Principe

Le FBM multi-octaves utilisé comme height map, avec une coloration par altitude. Plus d'octaves = plus de détail (rochers, graviers). C'est la base de tous les terrains procéduraux — de Minecraft à No Man's Sky.

Terrain en GLSL.

```
vec3 terrain(vec2 uv) {  
    float h = fbm(uv * 2.0);  
    if (h < 0.45) return vec3(0.15, 0.25, 0.5); // eau profonde  
    else if (h < 0.5) return vec3(0.20, 0.35, 0.6); // eau peu profonde  
    else if (h < 0.55) return vec3(0.85, 0.80, 0.6); // sable  
    else if (h < 0.65) return vec3(0.30, 0.50, 0.2); // plaine  
    else if (h < 0.75) return vec3(0.25, 0.40, 0.15); // forêt  
    else if (h < 0.85) return vec3(0.40, 0.35, 0.25); // montagne  
    else return vec3(0.95, 0.95, 0.98); // neige  
}
```

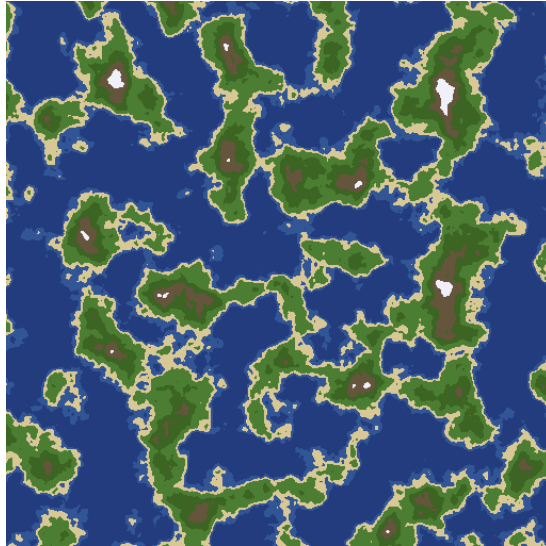


Figure 6: Texture de terrain procédurale — FBM avec coloration par altitude. Les seuils définissent les biomes : eau, sable, plaine, forêt, montagne, neige.

Autres effets

Effets avancés

- **Normal map procédurale** : on calcule les dérivées partielles du bruit ($\partial/\partial x$, $\partial/\partial y$) pour obtenir une normale perturbée — un relief procédural sans normal map pré-calculée.
- **Dplacement procédural** : le FBM déplace les sommets dans le vertex shader (voir le “Mode terrain” de la démo [session08_noise.html](#)).
- **Perturbation de domaine** : au lieu de perturber le **résultat** du bruit, on perturbe les **coordonnées** d’entrée : $\text{noise}(\text{uv} + \text{noise}(\text{uv} * 2) * 0.3)$. Crée des swirls et des déformations plus organiques.

Partie 2 : Render Targets

Rendre dans une texture au lieu de l’écran

Un **render target** est une texture dans laquelle le GPU rend la scène au lieu de l’afficher. Une fois la scène rendue, cette texture peut être :

- **Échantillonnée** par un autre shader (post-processing, réflexions).
- **Sauvée** dans un fichier (screenshots, lightmap baking).
- **Réutilisée** comme texture d’un autre objet (miroir, écran de surveillance).

Le pipeline en deux passes

1. **Passe 1** : rendre la scène (ou une partie) dans le render target.
2. **Passe 2** : rendre la scène finale à l’écran, en échantillonnant le render target là où on en a besoin.

C’est la base de **tous** les effets avancés : post-processing, shadow maps, réflexions, flou de profondeur.

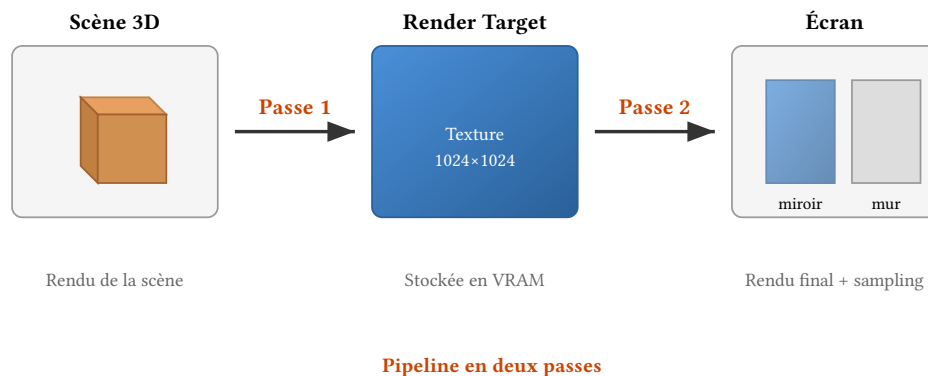


Figure 7: Pipeline en deux passes. Passe 1 : la scène est rendue dans une texture (render target) en VRAM. Passe 2 : la scène finale est rendue à l'écran en échantillonnant le render target (ici, pour un miroir).

Render target dans Babylon.js.

```
// 1. Créer un render target de 1024x1024
const rt = new BABYLON.RenderTargetTexture("mirrorRT", 1024, scene);
rt.renderList.push(...scene.meshes); // quels meshes rendre

// 2. L'assigner comme texture d'un matériau (ex: un miroir)
mirrorMat.diffuseTexture = rt;

// 3. Le rendu se fait automatiquement avant la frame principale
```

💡 Cas d'usage

- **Shadow maps** (vu en Session 5) : la scène est rendue du point de vue de la lumière dans un render target de profondeur.
- **Post-processing** (vu en Session 7 originale) : la scène est rendue dans un render target, puis un shader plein écran lit cette texture et applique un effet (bloom, flou, correction de couleur).
- **Réflexions temps réel** : rendre la scène depuis un miroir dans un render target, puis l'appliquer sur la surface du miroir.
- **Portal / écran** : rendre une vue dans un render target et l'afficher sur un mesh (écran de surveillance, tableau magique).

Exemple : miroir d'eau avec vagues

💬 Le cas d'usage classique

Un miroir d'eau combine **render target** et **textures procédurales** :

1. **Passe 1** : la scène est rendue depuis une **caméra de réflexion** (miroir de la caméra principale par rapport au plan d'eau) dans un render target.
2. **Passe 2** : la surface de l'eau échantillonne ce render target avec un **projective texturing**, et deux textures de vague animées en sens opposé déforment les UVs pour simuler les ondulations.

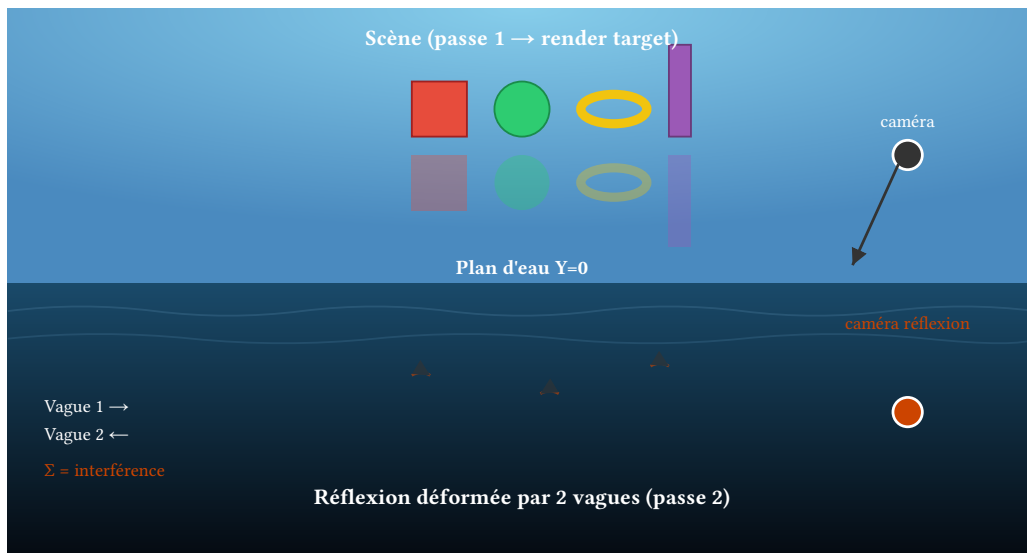


Figure 8: Miroir d'eau en deux passes. Passe 1 : la scène est rendue depuis la caméra de réflexion (miroir en Y) dans un render target. Passe 2 : la surface de l'eau échantillonne ce render target par projective texturing, avec deux vagues en sens opposé qui déforment les UVs — l'interférence crée un motif d'ondulation naturel.

Les deux textures de vague

L'effet d'ondulation utilise deux couches de bruit (FBM) qui défilent en **sens opposés** :

- **Vague 1** : `fbm(pos.xz * scale + vec2(time * speed, 0.0))` — défile vers $+x$.
- **Vague 2** : `fbm(pos.xz * scale * 1.3 - vec2(time * speed * 0.7, time * speed * 0.5))` — défile vers $-x$ et $-z$.

La **somme** des deux crée un motif d'**interférence** — comme deux vagues qui se croisent sur l'eau. Une seule couche donnerait un mouvement trop régulier ; deux couches en sens opposés donnent un motif chaotique naturel.

Projective texturing en GLSL.

```
// Transformer la position world-space en UVs du render target
vec4 projected = uReflectionVP * vec4(vWorldPos, 1.0);
vec3 projCoord = projected.xyz / projected.w; // NDC [-1, 1]
vec2 reflectUv = projCoord.xy * 0.5 + 0.5; // remap [0, 1]

// Distortion par les vagues (normale perturbée)
reflectUv += normal.xz * uDistortion;
reflectUv = clamp(reflectUv, 0.0, 1.0);

vec3 reflection = texture(uReflection, reflectUv).rgb;
```

`uReflectionVP` est la matrice view-projection de la **caméra de réflexion** — pas la caméra principale. C'est ce qui fait correspondre chaque fragment de l'eau au bon texel du render target.

💡 Démo interactive

Ouvrir `exemples/session08_water.html` : un miroir d'eau temps réel avec objets colorés qui se reflètent. La GUI contrôle la vitesse, fréquence et amplitude des vagues, la distortion de la réflexion, et la puissance du Fresnel. Observer :

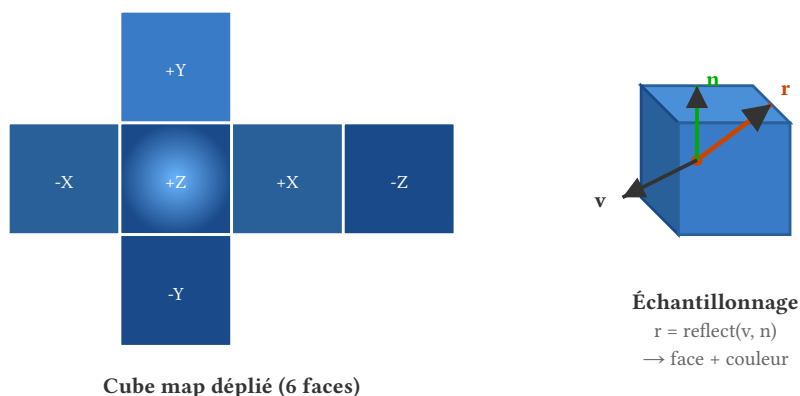
- De **biais** : la réflexion domine (Fresnel élevé).
- De **haut** : on voit la couleur de l'eau (Fresnel faible).
- Les **crêtes** des vagues sont plus claires, les **creux** plus sombres.
- Le **soleil** crée des glints spéculaires sur les crêtes.

Partie 3 : Cube Maps et Environment Maps

Le cube map

Un **cube map** est une texture 3D composée de 6 faces d'un cube (comme une boîte dépliée). On l'échantillonne avec une **direction 3D** $\vec{r}$ (pas des UVs 2D) — le sampler trouve la face correspondante et le texel dans cette face.

C'est la structure idéale pour représenter l'environnement **autour** d'un point : la direction $\vec{r}$ "regarde" dans une direction et obtient la couleur de l'environnement dans cette direction.



Cube map : 6 faces échantillonnées par direction 3D

Figure 9: Un cube map est composé de 6 faces (dépliées en croix). L'échantillonnage se fait avec un vecteur de direction 3D $\vec{r}$ — le sampler trouve la face et le texel correspondant. Pour les réflexions :

$$\vec{r} = \text{reflect}(\vec{v}, \vec{n}).$$

Skybox et réflexions PBR

- **Skybox** : un cube map placé autour de la scène, affiché en arrière-plan. Donne l'illusion d'un environnement lointain (ciel, montagnes, étoiles).
- **Réflexions PBR** : le matériau PBR (Session 5) réfléchit l'environnement. Pour un métal poli, la réflexion vient du cube map — le fragment shader calcule le vecteur de réflexion $\vec{r} = \text{reflect}(\vec{v}, \vec{n})$ et l'échantillonne dans le cube map.
- **IBL (Image Based Lighting)** : au lieu de lumières discrètes, l'éclairage vient de **tout** l'environnement — le cube map est préfiltré (diffuse irradiance + specular radiance) pour fournir l'éclairage diffus et spéculaire en un seul lookup.

💡 Créer un cube map dans Blender

Dans Blender : **World** → **Surface** → **Background** → **Color** → **Environment Texture**.

Charger un fichier HDR (ex: de HDRI Haven). Blender le convertit automatiquement en cube map pour l'éclairage et les réflexions.

Pour exporter un cube map depuis Blender : **Render** → **Bake Type** → **Cubemap**, ou utiliser un script Python pour rendre les 6 faces depuis un point donné.

Partie 4 : Texture Atlases et Arrays

Le problème des draw calls

Chaque matériau avec une texture différente = un **draw call** séparé. Un objet avec 10 matériaux = 10 draw calls. Pour réduire les draw calls, on **groupe** plusieurs textures en une seule.

4.A. Texture Atlas

Atlas

Un **atlas** est une grande texture qui contient plusieurs sous-textures côte à côte. Les UVs de chaque objet pointent dans la région de l'atlas qui contient sa texture. Un seul draw call pour tous les objets qui utilisent l'atlas.

- **Avantage** : un draw call pour N matériaux.
- **Inconvénient** : le wrapping (repeat) ne marche plus — les UVs doivent rester dans la région de la sous-texture, sinon on "déborde" sur la voisine.

4.B. Texture Array

Texture Array

Un **texture array** est une pile de textures de **même taille**, échantillonnées avec un index de couche en plus des UVs : `texture(samplerArray, vec3(uv, layer))`. Plus moderne que l'atlas, pas de problème de wrapping.

- **Avantage** : un draw call, wrapping préservé.
- **Inconvénient** : toutes les textures doivent avoir la même résolution et le même format.

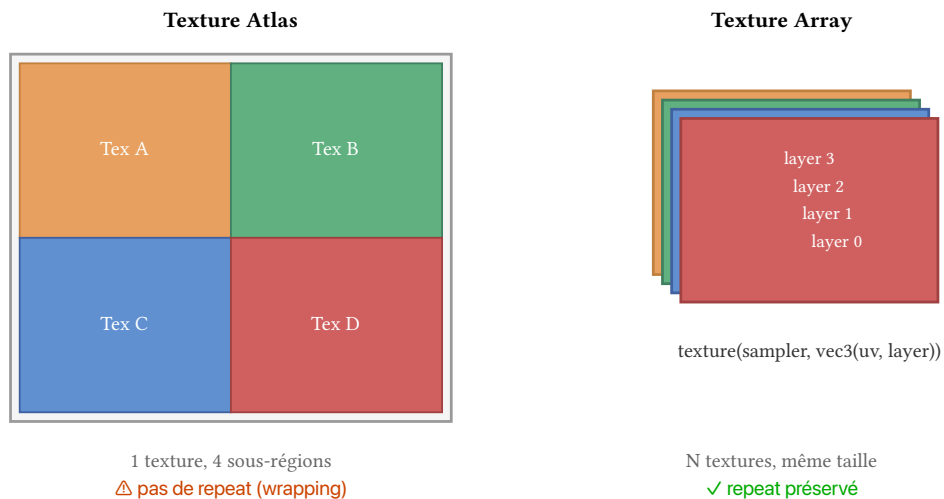


Figure 10: Texture atlas (gauche) : une grande texture découpée en sous-régions — pas de wrapping. Texture array (droite) : une pile de textures de même taille échantillonnées par index de couche — wrapping préservé.

4.C. Trim Sheets

! Le problème des éléments répétitifs

Dans un environnement (bâtiments, intérieurs, véhicules), beaucoup d'éléments sont des **bandes** répétitives : plinthes, moulures, bordures, gouttières, cadres. Faire une texture complète par élément = dizaines de draw calls et de la VRAM gaspillée. Le **trim sheet** regroupe toutes ces bandes dans **une seule texture**.

Principe du trim sheet

Un **trim sheet** est une texture (souvent **tall** — plus haute que large) qui contient plusieurs **bandes horizontales** empilées verticalement. Chaque bande est un matériau ou un élément d'architecture :

- plinthe basse, moulure, frise, tableau, corniche, etc.
- ou : bordure de toit, gouttière, cadre de fenêtre, rail.

Le mesh est **déplié** de façon à ce que chaque partie (plinthe, mur, moulure) utilise une **bande horizontale** spécifique de la texture. Les UVs horizontaux se **répètent** le long de la bande (le motif est tilable en X), tandis que les UVs verticaux sélectionnent la bonne bande.

$$UV_x \text{ in bande}_{\text{début}} \dots \text{bande}_{\text{fin}}, \quad UV_y \text{ in } [0, 1] \text{ répété}$$

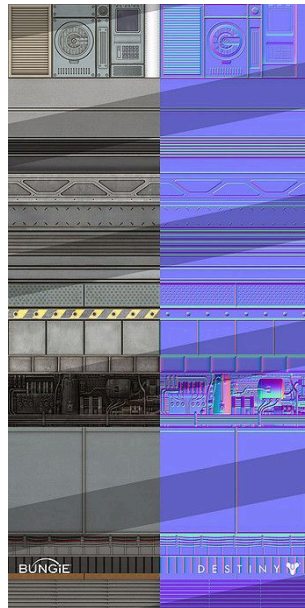


Figure 11: Exemple de trim sheet : plusieurs bandes de matériaux (bois, métal, peinture) empilées verticalement. Chaque bande est tilable horizontalement — on peut répéter le motif le long d'un mur sans voir de seam.

💡 Avantages du trim sheet

- **Un seul draw call** pour tout un bâtiment (murs, plinthes, moulures, cadres).
- **Wrapping horizontal préservé** — les bandes sont tilable en X, on peut répéter le motif autant de fois que nécessaire.
- **PBR complet** : le trim sheet contient souvent les **4 maps** (albedo, normal, roughness, AO) empilées de la même façon, synchronisées.
- **Workflow modulaire** : on construit des bibliothèques de trims réutilisables entre projets.

⚠️ Contraintes

- Le **dépliage** doit être précis : chaque partie du mesh doit aligner ses UVs sur la bonne bande. C'est un travail de modélisation rigoureux.
- Les bandes ont une **hauteur fixe** en pixels — si une plinthe fait 10cm dans le monde réel, sa bande fait par exemple 64px, et il faut que la résolution soit suffisante pour le détail.
- Le wrapping **vertical** ne marche pas (comme pour l'atlas) — on ne peut répéter qu'horizontalement.

Trim sheet vs Atlas vs Array

Critère	Atlas	Trim Sheet	Texture Array
Forme	Grille 2D	Bandes verticales	Pile de couches
Wrapping	Aucun	Horizontal seul	Complet
Taille des sous-textures	Variable	Hauteur fixe	Identique

Critère	Atlas	Trim Sheet	Texture Array
Usage typique	UI, sprites	Architecture, environnements	Terrain, matériaux

Le trim sheet est le **compromis** entre l'atlas (pas de wrapping) et l'array (tout est identique) : il préserve le wrapping dans **une direction** — celle qui compte pour les éléments linéaires.

Partie 5 : UV Avancés — Triplanar Mapping

🗨 Le problème des UVs sur les formes complexes

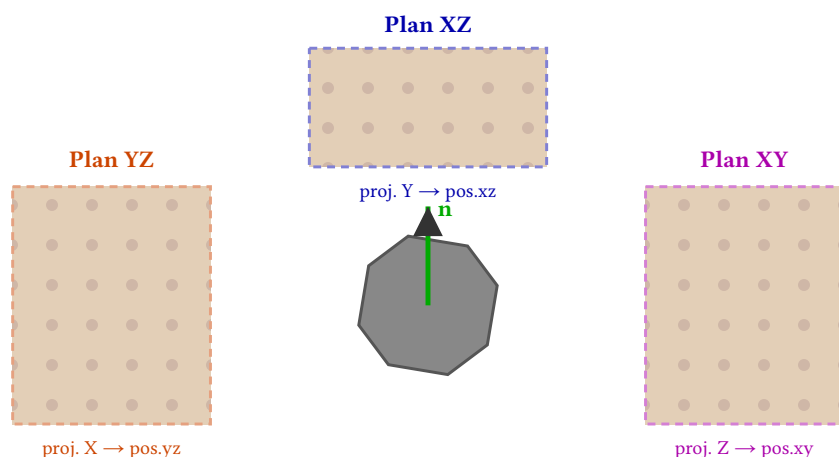
Les UVs classiques nécessitent un **dépliage** (Session 7) — un processus manuel, fastidieux, et qui produit des seams. Pour les formes **naturelles** (terrain, rochers, organique), le dépliage est souvent impraticable. Le **triplanar mapping** résout ce problème en **supprimant les UVs**.

Principe du triplanar mapping

Au lieu d'utiliser un canal UV, on **projette** la texture sur 3 plans orthogonaux (X, Y, Z) et on **mélange** les trois projections selon la normale de la surface :

- Une face orientée vers $+Y$ utilise surtout la projection sur le plan XZ (vue de dessus).
- Une face orientée vers $+X$ utilise surtout la projection sur le plan YZ (vue de côté).
- Le mélange est pondéré par les composantes de la normale : $w_i = |n_i|^p$ (exposant p pour durcir la transition).

$$\text{color} = \frac{w_x \cdot \text{proj}_x + w_y \cdot \text{proj}_y + w_z \cdot \text{proj}_z}{w_x + w_y + w_z}$$



$$w = |n|^p \rightarrow \text{color} = (w_x \cdot \text{proj}_x + w_y \cdot \text{proj}_y + w_z \cdot \text{proj}_z) / (w_x + w_y + w_z)$$

La normale détermine quelle projection domine

Figure 12: Triplanar mapping : la texture est projetée sur 3 plans orthogonaux (YZ, XZ, XY). Le poids de chaque projection est déterminé par la normale de la surface $w_i = |n_i|^p$. Une face orientée vers le haut utilise la projection XZ ; une face latérale utilise YZ ou XY.

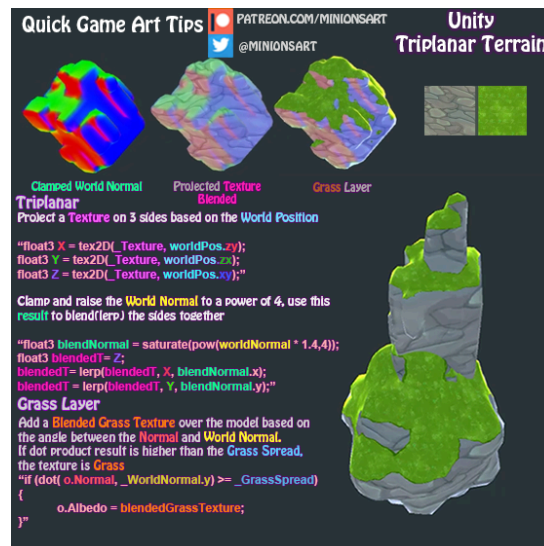


Figure 13: Exemple de triplanar mapping sur une forme complexe : la même texture est projetée depuis 3 directions, et les transitions sont fondues selon la normale — pas de seam visible, pas de dépliage nécessaire.

Triplanar mapping en GLSL (simplifié).

```
uniform sampler2D uTexture;
in vec3 vWorldPos;
in vec3 vNormal;

vec3 triplanar(sampler2D tex, vec3 pos, vec3 normal, float scale) {
    // 3 projections avec les coordonnées de position
    vec3 cx = texture(tex, pos.yz * scale).rgb; // plan YZ
    vec3 cy = texture(tex, pos.xz * scale).rgb; // plan XZ
    vec3 cz = texture(tex, pos.xy * scale).rgb; // plan XY

    // Poids selon la normale (power pour durcir les transitions)
    vec3 w = pow(abs(normal), vec3(8.0));
    w /= (w.x + w.y + w.z);

    return w.x * cx + w.y * cy + w.z * cz;
}
```

💡 Dans le Node Material Editor

Babylon.js NME propose un block **Triplanar** qui fait ce calcul automatiquement. On branche une texture, la position world-space et la normale — le block sort la couleur mélangée. Parfait pour les terrains, rochers, et toute géométrie sans UVs.

Partie 6 : Parallax Mapping

Le problème des normal maps

Les normal maps (Session 7) simulent du relief en **truquant l'éclairage**. Mais la surface reste plate — si on se déplace latéralement, les détails ne bougent **pas** avec la parallaxe. L'illusion se brise dès qu'on bouge la caméra.

Parallax mapping — décaler les UVs

Le **parallax mapping** utilise une **height map** (grayscale) pour décaler les UVs selon l'angle de vue. L'idée : si on regarde la surface en biais, les zones “hautes” devraient apparaître décalées par rapport aux zones “basses” — comme si on regardait un relief réel en biais.

$$UV_{\text{décalé}} = UV + \text{height}(UV) \cdot (\vec{v}_{\text{tangent}} \cdot k)$$

où $\vec{v}_{\text{tangent}}$ est le vecteur de vue dans l'espace tangent et k est un facteur d'intensité.

Parallax Occlusion Mapping (POM)

Le **Parallax Occlusion Mapping** est la version avancée : au lieu d'un seul décalage, on **ray-marche** dans la height map — on avance par petits pas le long du vecteur de vue, on compare la hauteur à chaque pas, et on s'arrête quand on “touche” la surface. Résultat : les zones hautes **occluent** les zones basses — un vrai relief apparent, avec des silhouettes déformées.

Le coût : plusieurs échantillons de texture par fragment (8 à 32 typiquement). Plus précis que le parallax simple, mais plus cher.

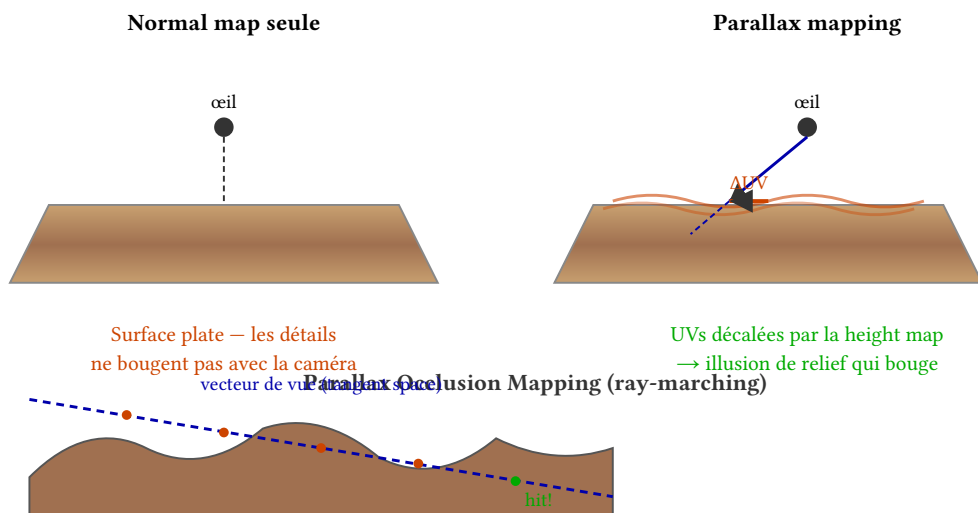


Figure 14: Normal map seule (gauche) : la surface reste plate, les détails ne bougent pas avec la caméra. Parallax mapping (droite) : les UVs sont décalées par la height map selon l'angle de vue — le relief apparent se déplace. En bas : le POM ray-marche dans la height map jusqu'à toucher la surface (point vert).

Parallax mapping simple en GLSL.

```
uniform sampler2D uHeightMap;
in vec2 vUV;
in vec3 mViewDirTangent; // vecteur de vue dans l'espace tangent

vec2 parallaxUV(vec2 uv, vec3 viewDir) {
    float height = texture(uHeightMap, uv).r; // [0,1]
    vec2 offset = viewDir.xy * height * 0.1; // 0.1 = intensité
    return uv - offset;
}
```

```
void main() {
    vec2 uv = parallaxUV(vUV, vViewDirTangent);
    vec3 albedo = texture(uAlbedoMap, uv).rgb; // utiliser les UVs décalées
    // ...
}
```

💡 Dans le Node Material Editor

Babylon.js NME propose un block **Parallax Occlusion Mapping**. On branche la height map, la normal map, et le vecteur de vue — le block sort les UVs décalées à utiliser pour toutes les autres textures du set PBR.

⚠️ Limites du parallax

- La silhouette reste plate — le parallax **truque** la surface, pas les bords. Vu de profil très rasant, l'illusion se brise.
- Le POM coûte plusieurs échantillons par fragment — à réserver pour les surfaces proches de la caméra.
- Pour un vrai relief de silhouette, il faut du **displacement mapping** (déplacement des sommets) — beaucoup plus coûteux.

Partie 7 : Detail Textures et Blending

Le problème de la résolution

Une texture de sol 2048×2048 couvrant 10×10 mètres a une densité de ~ 200 texels/mètre. De près, chaque texel couvre ~ 5 “mm” — c’est flou. Augmenter la résolution à 8K coûte 16× plus de VRAM. Solution : **superposer** une texture de détail.

Detail texture

On superpose deux textures :

- **Base** : la texture unique du matériau (2048×2048), avec ses couleurs et ses variations.
- **Détail** : une petite texture tiling (512×512) répétée 10×, en **grayscale** ou **normal only**, qui ajoute du grain fin (pores, rayures, micro-relief).

Le mélange se fait par **multiplication** (pour l’albedo) ou **addition** (pour les normales) :

$$\text{albedo}_{\text{final}} = \text{albedo}_{\text{base}} \cdot (0.5 + \text{detail} \cdot 0.5)$$

Le facteur 0.5 centre le détail autour de 1.0 pour ne pas assombrir/éclaircir globalement.

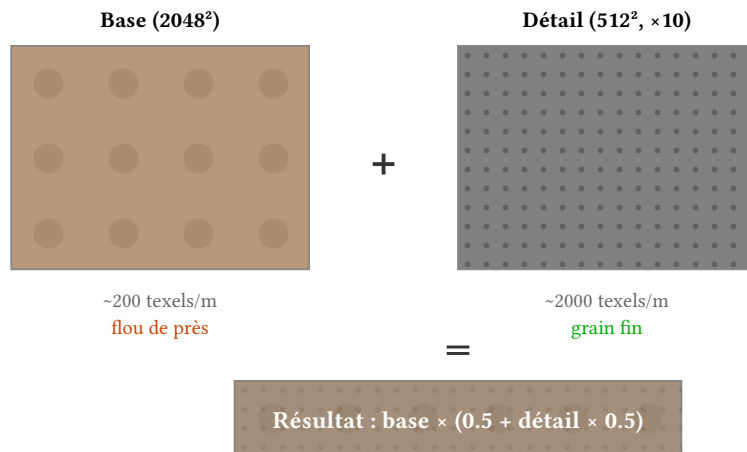


Figure 15: Detail texture : la base (2048^2 , 200 texels/m) est floue de près. On superpose une petite texture de détail (512^2 , répétée $10\times$) qui ajoute du grain fin (2000 texels/m). Le mélange par multiplication préserve les couleurs de base tout en ajoutant du micro-relief.

💡 Dans Blender — mix nodes

Dans le Shader Editor de Blender, utiliser un **Mix RGB** node en mode **Multiply** pour mélanger la base et le détail. Le **Fac** contrôle l'intensité du détail. Pour les normales, utiliser un **Normal Map** node avec la base, puis un **Mix** node en mode **Add** avec le détail normal.

Partie 8 : Pratique

💡 Objectif pratique

Mettre en place un tri-planar mapping avec une texture de détail.

Bonus commun

- **Triplanar** : remplacer les UVs classiques par un block **Triplanar** sur un terrain ou un rocher. Observer l'absence de seams.
- **Texture blending** : mélanger deux sets PBR (pierre + mousse) avec un masque procédural (bruit seillé). C'est la technique utilisée pour les terrains avec plusieurs matériaux (herbe, terre, rocher).